Recursive Least Squares for Online Parameter Estimation

1.0 Learning Outcomes.....1
2.0 Introduction.....1
3.0 From Batch to Recursive Least Squares.....2
4.0 The Forgetting Factor.....4
5.0 Practical Considerations.....6
6.0 Example: Tracking a Change in Motor Gain (Simulation).....7
7.0 Summary.....8
References.....9
Recommended Citation.....9

1.0 Learning Outcomes

After studying this ACE-Lab learning guide, you should be able to demonstrate the Learning Outcomes listed in Table 1. On completion of the ACE-Lab learning guide, revisit Table 1 and confirm whether each Learning Outcome has been achieved.

Table 1: *Learning Outcomes from the Recursive Least Squares for Online Parameter Estimation Learning Guide*

ID	Learning Outcome	Achieved
1.	Explain why a model identified once, offline, may no longer describe a system whose behaviour changes during operation.	☐
2.	Derive the recursive least squares (RLS) update from the batch linear least squares (LLS) estimate and explain the role of the prediction error, the gain vector and the covariance matrix.	☐
3.	Explain how the forgetting factor sets the memory of the estimator, and choose a value that balances tracking speed against sensitivity to noise.	☐
4.	Identify the practical requirements for reliable online estimation, including initialisation, excitation, covariance wind-up and offsets caused by friction.	☐
5.	Use MATLAB to estimate a first-order model recursively and track a change in the gain of a DC motor.	☐

2.0 Introduction

The Linear Least Squares for Parameter Estimation Learning Guide showed how a model of a system can be found from a block of measured input and output data. The data were collected first, and the parameters were then calculated in a single step, offline. This works well when the system does not change. A physical system, however, rarely stays the same: the load on a motor changes, its bearings wear, its temperature rises and its battery voltage falls. A model identified on the day the system was commissioned may be wrong a week later, or even a minute later. An important question therefore arises:

How can the parameters of a model be estimated, and kept up to date, while the system is running?

The answer explored in this Learning Guide is recursive least squares (RLS). Rather than solving the least squares problem again for all of the data at every sample, RLS updates the previous estimate using only the newest measurement. The calculation is small, takes the same time at every sample and needs no matrix inversion, so it can run in real time on a microcontroller. With a forgetting factor, RLS gives more weight to recent data than to old data, which allows the estimate to follow a system whose parameters change.

Online estimation is the first ingredient of adaptive control. Once a model is available at every sample, it can be used to recalculate the controller gains automatically, which is the subject of the Adaptive Control: the Self-Tuning PI Controller Learning Guide.

Why is Recursive Least Squares Important?

RLS is important in control engineering because:

i. RLS estimates the model parameters while the system is operating, without storing all of the past data.

ii. With a forgetting factor, the estimate tracks changes in the system, such as a change in load, wear or supply voltage.

iii. Each update needs only a few multiplications and additions, so it can run at every sample on low-cost hardware such as the Arduino.

iv. The estimated parameters can be used to monitor the condition of a system, to detect faults, or to recalculate the gains of a controller in an adaptive control system.

3.0 From Batch to Recursive Least Squares

The Model and the Batch Estimate

As in the Linear Least Squares for Parameter Estimation Learning Guide, the system is described by a difference equation. For a first-order model:

$$y_k = -a_1 y_{k-1} + b_0 u_{k-1} + e_k \tag{1}$$

where u_k is the input, y_k the measured output and e_k the measurement noise. The equation can be written as the product of a regressor vector, φ_k, which contains the past outputs and inputs, and a parameter vector, θ:

$$y_k = \varphi_k^T \theta + e_k, \qquad \varphi_k = \begin{bmatrix} -y_{k-1} \\ u_{k-1} \end{bmatrix}, \qquad \theta = \begin{bmatrix} a_1 \\ b_0 \end{bmatrix} \tag{2}$$

Stacking k observations gives $y = \Phi\theta$, where each row of the observation matrix Φ is a regressor φ_i^T, and the LLS estimate after k samples is:

$$\hat{\theta}_k = \left[\Phi_k^T \Phi_k\right]^{-1} \Phi_k^T y_k = \left[\sum_{i=1}^{k} \varphi_i \varphi_i^T\right]^{-1} \sum_{i=1}^{k} \varphi_i y_i \tag{3}$$

Equation (3) could be recalculated every time a new sample arrives, but the work grows with the amount of data and a matrix must be inverted at every sample. RLS avoids both problems.

The Recursive Update

Define the covariance matrix as the inverse of the information matrix:

$$P_k = \left[\Phi_k^T \Phi_k\right]^{-1}, \qquad P_k^{-1} = P_{k-1}^{-1} + \varphi_k \varphi_k^T \tag{4}$$

The second form shows that each new sample simply adds $\varphi_k \varphi_k^T$ to the information already collected. Substituting Equation (4) into Equation (3), and using the matrix inversion lemma to update P_k without inverting a matrix, gives the RLS algorithm (Åström and Wittenmark, 1995; Ljung, 1999). At each sample, the four steps are:

$$\varepsilon_k = y_k - \varphi_k^T \hat{\theta}_{k-1} \tag{5}$$

$$L_k = \frac{P_{k-1}\varphi_k}{1 + \varphi_k^T P_{k-1} \varphi_k} \tag{6}$$

$$\hat{\theta}_k = \hat{\theta}_{k-1} + L_k \varepsilon_k \tag{7}$$

$$P_k = P_{k-1} - L_k \varphi_k^T P_{k-1} \tag{8}$$

The structure of the algorithm is shown in Figure 1, and the meaning of each term is summarised in Table 2.

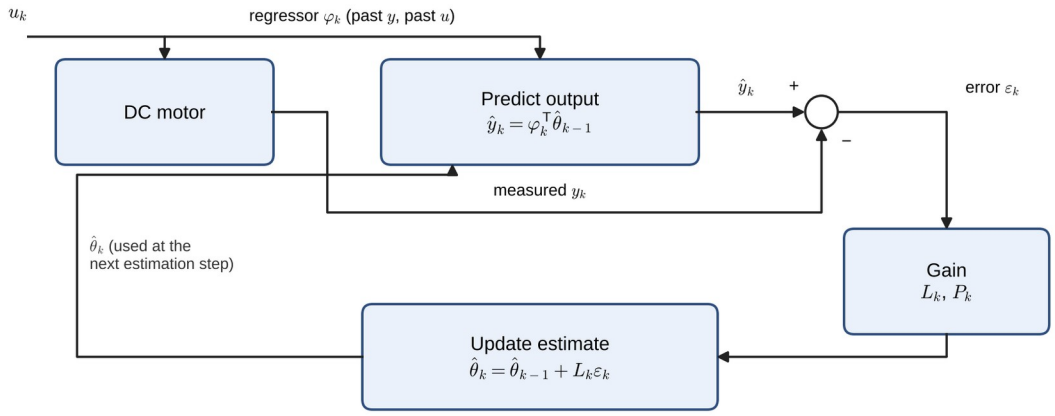


Figure 1: *The Recursive Least Squares Estimator: Predict, Compare, and Correct the Estimate at Every Sample*

Table 2: *Terms in the Recursive Least Squares Update*

Term	Meaning
$\varphi_k^T \hat{\theta}_{k-1}$, prediction	The output the current model expects, using the previous estimate
ε_k, prediction error	How far the measurement differs from the prediction: the new information in the sample
L_k, gain vector	How strongly each parameter is corrected by the prediction error
P_k, covariance matrix	How uncertain the estimate is: large P_k gives large corrections, small P_k gives small ones

Equation (7) has an intuitive meaning: the new estimate is the old estimate plus a correction proportional to the prediction error. If the model predicts the measurement perfectly, $\varepsilon_k = 0$ and the estimate is not changed. When the estimate is uncertain (large P_k) it is corrected strongly; as data accumulate, P_k

becomes smaller and each new sample changes the estimate less. With the same data, and a large initial P_0, RLS gives the same estimate as the batch LLS solution of Equation (3).

Example 1: One step of the recursive least squares algorithm

A first-order motor model, sampled every T_e = 0.05 s, has the current estimate and covariance matrix

$$\hat{\theta}_{k-1} = \begin{bmatrix} -0.8 \\ 2.0 \end{bmatrix}, \qquad P_{k-1} = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}$$

The previous speed was y_{k-1} = 50 RPM, the previous voltage u_{k-1} = 5 V, and the new measured speed is y_k = 52 RPM. Calculate the updated estimate using a forgetting factor λ = 0.98 (Section 4.0), and the gain and time constant of the motor before and after the update.

Solution:

The regressor is $\varphi_k = [-50 \;\; 5]^T$. The prediction and the prediction error are:

$$\varphi_k^T \hat{\theta}_{k-1} = (-50)(-0.8) + (5)(2.0) = 50 \text{ RPM}, \qquad \varepsilon_k = 52 - 50 = 2 \text{ RPM}$$

Next, $P_{k-1}\varphi_k = [-500 \;\; 50]^T$ and $\varphi_k^T P_{k-1}\varphi_k$ = 25 000 + 250 = 25 250, so the gain vector is:

$$L_k = \frac{1}{0.98 + 25\,250} \begin{bmatrix} -500 \\ 50 \end{bmatrix} = \begin{bmatrix} -0.01980 \\ 0.00198 \end{bmatrix}$$

and the updated estimate is:

$$\hat{\theta}_k = \begin{bmatrix} -0.8 \\ 2.0 \end{bmatrix} + 2 \begin{bmatrix} -0.01980 \\ 0.00198 \end{bmatrix} = \begin{bmatrix} -0.8396 \\ 2.0040 \end{bmatrix}$$

Using $K = b_0/(1+a_1)$ and $\tau = -T_e/\ln(-a_1)$ from the Linear Least Squares for Parameter Estimation Learning Guide, the model changes from K = 10.0 RPM/V and τ = 0.224 s to K = 12.5 RPM/V and τ = 0.286 s. The motor ran 2 RPM faster than predicted, so the estimated gain has increased. Only multiplications, additions and one division were needed.

4.0 The Forgetting Factor

Weighting Recent Data

The batch estimate and Equations (5) to (8) give every sample the same weight, however old it is. After a long time, P_k becomes very small, the gain L_k approaches zero and the estimate effectively stops changing. This is ideal for a system that never changes, but it means that the estimator cannot follow a change in the system.

The solution is to minimise a weighted cost in which older errors count for less:

$$J_k = \sum_{i=1}^{k} \lambda^{k-i} \left(y_i - \varphi_i^T \theta\right)^2, \qquad 0 < \lambda \leq 1 \tag{9}$$

where λ is the forgetting factor. A sample that is n steps old is weighted by λ^n. With forgetting, Equations (6) and (8) become:

$$L_k = \frac{P_{k-1}\varphi_k}{\lambda + \varphi_k^T P_{k-1}\varphi_k}, \qquad P_k = \frac{1}{\lambda}\left(P_{k-1} - L_k \varphi_k^T P_{k-1}\right) \tag{10}$$

Dividing by $\lambda < 1$ stops P_k from shrinking to zero, so the estimator stays alert to new information. The forgetting factor sets the memory of the estimator, that is, roughly how many samples, or how many seconds, of data the estimate is based on:

$$N_{mem} \approx \frac{1}{1-\lambda} \text{ samples}, \qquad T_{mem} \approx \frac{T_e}{1-\lambda} \text{ seconds} \tag{11}$$

where T_e is the interval between estimates. Typical values are given in Table 3 for T_e = 0.05 s, the estimation interval used on the ACE-Lab.

Table 3: *Memory of the Estimator for Different Forgetting Factors* (T_e = 0.05 s)

Forgetting factor, λ	Memory (samples)	Memory (s)	Behaviour
1	All data	All data	No forgetting: best for a system that does not change
0.999	1000	50	Very smooth estimate; follows only slow changes
0.995	200	10	Smooth estimate; follows changes within tens of seconds
0.99	100	5	A common compromise for a motor whose load changes
0.95	20	1	Follows changes quickly, but the estimate is noisy

Choosing the Forgetting Factor

Choosing λ is a compromise. A short memory (smaller λ) allows the estimate to follow a change quickly, but the estimate is based on fewer samples, so it is more affected by measurement noise. A long memory (larger λ) averages out the noise, but the estimate responds slowly when the system changes. A useful starting point is to choose a memory that is short compared with how quickly the system is expected to change, but long compared with the time constant of the system, so that each memory contains several responses to changes in the input.

Covariance Wind-Up

Forgetting has one important side effect. If the input does not change, for example when a motor runs at a constant speed for a long time, the regressor carries no new information about the dynamics, and $\varphi_k^T P_{k-1} \varphi_k$ remains small. Equation (10) then divides P_k by λ at every sample, so P_k grows exponentially. This is known as covariance wind-up. When the input eventually changes, the very large P_k produces a very large correction and the estimate can jump to unrealistic values. Common safeguards are:

- Limiting the size of P_k, for example by setting $\lambda = 1$ whenever the trace of P_k (the sum of its diagonal elements) exceeds a limit.
- Updating the estimate only when the input is changing enough to excite the system (dead-zone or conditional updating).
- Keeping the system excited, for example by using a reference that changes regularly.

The ACE-Lab estimators use the first approach: λ is set to 1 whenever trace(P_k) > 10^4.

5.0 Practical Considerations

Initialisation

RLS needs an initial estimate, $\hat{\theta}_0$, and an initial covariance matrix, $P_0 = p_0 I$, where I is the identity matrix. A large p_0 (for example 100 to 1000) says that the initial estimate is very uncertain, so the first measurements correct it quickly. A small p_0 says that the initial estimate is trusted, so it changes slowly. When a reasonable guess is available, for example from an earlier LLS identification, it should be used as $\hat{\theta}_0$, with a moderate p_0. In closed-loop operation, a poor initial estimate can produce poor controller gains during the first few seconds, so it is good practice to use fixed, safe gains until the estimate has settled.

Excitation

As with LLS, the input must change enough to excite the dynamics of the system. If the input and output are constant, every regressor is the same, and the data contain no information about the time constant. RLS cannot then separate a_1 from b_0, and the estimate may drift. In an identification experiment this is achieved with a changing input, such as the sum of two square waves used on the ACE-Lab. In a control system, regular changes in the reference provide the excitation.

Offsets: Friction and Dead-Band

A real DC motor does not start to turn until the voltage exceeds a dead-band caused by static friction, and the encoder speed is never negative. Around an operating point, this behaves as a constant offset, which the model of Equation (1) cannot represent: the estimate of b_0, and hence the gain, is then biased. The offset is included by adding a constant term, c, to the model, and a constant 1 to the regressor:

$$y_k = -a_1 y_{k-1} + b_0 u_{k-1} + c \tag{12}$$

with the regressor and parameter vectors:

$$\varphi_k = \begin{bmatrix} -y_{k-1} & u_{k-1} & 1 \end{bmatrix}^T, \qquad \theta = \begin{bmatrix} a_1 & b_0 & c \end{bmatrix}^T \tag{13}$$

The gain and time constant are still found from a_1 and b_0, and the dead-band voltage is estimated as $V_{\text{dead}} = -c/b_0$.

Estimation Interval and Filtering

The estimate does not have to be updated at every control sample. On the ACE-Lab, the controller runs every $T_s = 0.01$ s, but the motor model is estimated every $T_e = 5T_s = 0.05$ s, using the average voltage applied over each interval. At 0.01 s, the encoder resolves the speed only in steps of about 1.68 RPM, so the change in speed from one sample to the next is mostly quantisation noise; over 0.05 s the motor dynamics are much clearer. The measured speed is also smoothed with a low-pass filter. If the speed is filtered but the voltage is not, the lag of the filter appears in the estimated time constant. Passing the voltage through the same filter as the speed removes this bias, because the same filter then acts on both sides of the difference equation.

6.0 Example: Tracking a Change in Motor Gain (Simulation)

In this example, a first-order DC motor with K = 16 RPM/V and τ = 0.25 s is simulated with measurement noise. At t = 30 s a load is added, and the gain of the motor falls to 60% of its original value. The input is the sum of two square waves, which excites the motor throughout the test. The same data are processed by three RLS estimators with λ = 1, 0.99 and 0.95, so that the effect of the forgetting factor can be compared directly. No Simulink model is needed.

MATLAB Script: tracking a change in the gain of a DC motor using recursive least squares

```
clear; clc; close all;

%% Simulate a first-order DC motor whose gain changes at t = 30 s
Te = 0.05;                          % estimation interval [s]
T  = 60;  N = round(T/Te);  t = (0:N-1)'*Te;
K_true = 16*ones(N,1);              % motor gain [RPM/V]
K_true(t >= 30) = 16*0.6;           % load added at t = 30 s
tau = 0.25;  p = exp(-Te/tau);      % time constant [s], discrete pole
u = 4 + 1.0*sign(sin(2*pi*t/3.0)) ...
      + 0.6*sign(sin(2*pi*t/7.4)); % input voltage [V]
u(t < 1) = 0;
y = zeros(N,1);
for k = 2:N
    y(k) = p*y(k-1) + K_true(k-1)*(1 - p)*u(k-1);
end
y = y + 0.5*randn(N,1);             % measurement noise [RPM]

%% Recursive least squares with three forgetting factors
lambdas = [1 0.99 0.95];
K_est = zeros(N,3);  tau_est = nan(N,3);
for j = 1:3
    lambda = lambdas(j);
    theta = [0; 0];  P = 1000*eye(2);          % initial estimate
    for k = 2:N
        phi   = [-y(k-1); u(k-1)];             % regressor
        e     = y(k) - phi'*theta;             % prediction error
        L     = P*phi/(lambda + phi'*P*phi);   % gain vector
        theta = theta + L*e;                   % update the estimate
        P     = (P - L*phi'*P)/lambda;         % update the covariance
        K_est(k,j) = theta(2)/(1 + theta(1)); % gain [RPM/V]
        if -theta(1) > 0 && -theta(1) < 1
            tau_est(k,j) = -Te/log(-theta(1)); % time constant [s]
        end
    end
end

%% Plot the data and the estimates
subplot(311); plot(t, y, t, 10*u, 'k'); grid on; ylabel('Signal');
legend('Measured speed, y [RPM]', 'Voltage x10, u [V]');
subplot(312); plot(t, K_est, t, K_true, 'k--'); grid on;
ylabel('K (RPM/V)');
legend('\lambda = 1', '\lambda = 0.99', '\lambda = 0.95', 'True K');
subplot(313); plot(t, tau_est); yline(tau, 'k--'); grid on;
ylabel('\tau (s)'); xlabel('Time (s)'); ylim([0 0.5]);
```

Results: the estimates are shown in Figure 2 (your results will differ slightly, because the measurement noise is random). Before the load is added, all three estimators converge to $K \approx 16$ RPM/V and $\tau \approx 0.25$ s within a few seconds. After the change, the estimator with λ = 0.95 (memory 1 s) settles within 10% of the new gain of 9.6 RPM/V in about 1.5 s, and λ = 0.99 (memory 5 s) in about 11 s. Without forgetting (λ = 1), the estimate moves only slowly from 16 RPM/V, because it is an average of all of the data, before and after the change: after 30 s it is still about 13.4 RPM/V, and the estimated time constant is wrong. The price of the short memory is noise: before the change, the standard deviation of the

estimated gain is about 0.01 RPM/V with λ = 0.99 but about 0.04 RPM/V with λ = 0.95.

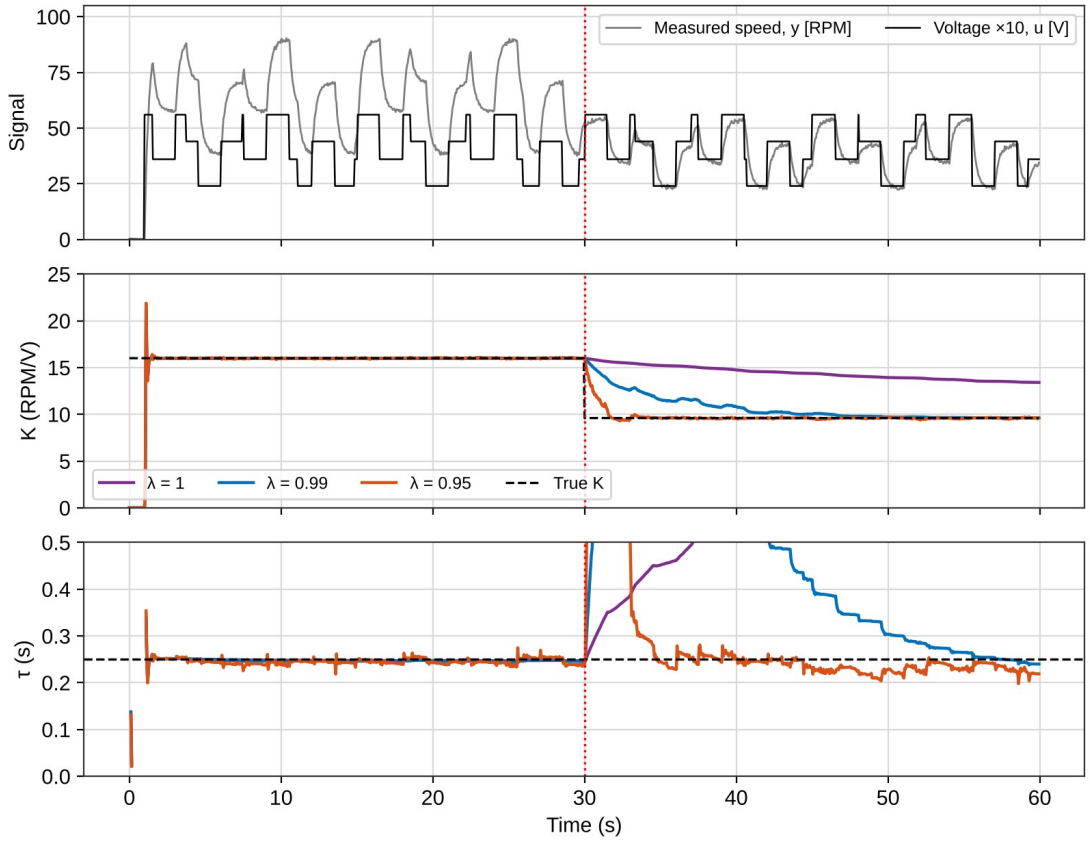


Figure 2: *Simulated Data (Top), and the Estimated Gain (Middle) and Time Constant (Bottom) for Three Forgetting Factors; the Motor Gain Falls to 60% at t = 30 s*

Further Investigation

Investigate how the forgetting factor, the noise and the excitation affect the estimate:

- **Forgetting factor:** repeat the estimation for λ = 0.999, 0.995 and 0.98. For each value, record the time taken for the estimated gain to settle within 10% of its new value, and the standard deviation of the estimate between 20 s and 30 s. Plot both against the memory $T_e/(1-\lambda)$ and explain the trade-off.
- **Noise:** increase the noise from 0.5 RPM to 2 RPM and 5 RPM. How does this change your choice of λ?
- **Excitation:** replace the input with a constant 5 V from t = 20 s. Plot the trace of P against time for λ = 0.99 and explain what happens. Add a limit that sets λ = 1 whenever trace(P) > 10^4, and repeat.
- **Initialisation:** repeat with $P_0 = I$ and $P_0 = 10^5 I$, and with the initial estimate $\hat{\theta}_0 = [-0.8 \;\; 2.0]^T$. How long does each estimator take to converge?

7.0 Summary

This Learning Guide extended linear least squares (LLS) to recursive least squares (RLS), which updates the estimate of a model at every sample using only the newest measurement. The new estimate is the old estimate plus a correction

proportional to the prediction error, $\hat{\theta}_k = \hat{\theta}_{k-1} + L_k\varepsilon_k$, where the gain vector L_k depends on the covariance matrix P_k, which describes how uncertain the estimate is. No matrix inversion is needed, so RLS can run in real time on a microcontroller.

A forgetting factor, λ, gives older data less weight and sets the memory of the estimator to about $T_e/(1-\lambda)$ seconds. This allows the estimate to follow a system whose parameters change, at the cost of greater sensitivity to noise. Practical estimators also need a sensible initial estimate, an input that excites the system, protection against covariance wind-up, and a constant term to account for offsets such as the dead-band of a DC motor. The key progression introduced in this guide can be summarised as:

new measurement → prediction error → gain vector → updated estimate → $\hat{K}$, $\hat{\tau}$ at every sample

The online estimate forms the basis of the self-tuning controller in the Adaptive Control: the Self-Tuning PI Controller Learning Guide, and is put into practice on the ACE-Lab in the practical exercise, Online Parameter Estimation of a DC Motor using Recursive Least Squares.

References

Åström, K. J. and Wittenmark, B. (1995). *Adaptive Control*, 2nd edn. Reading, MA: Addison-Wesley.

Ljung, L. (1999). *System Identification: Theory for the User*, 2nd edn. Upper Saddle River, NJ: Prentice Hall.

Pickering, J. E. (2026). Linear Least Squares for Parameter Estimation. ACE-Lab. Teaching Notes: ACE-Lab Learning Guide, Version 2.0. Zenodo. https://doi.org/10.5281/zenodo.23568450

Recommended Citation

If you have used this learning guide in your teaching, learning or research, please cite it using the reference below, adapting the format as required by your preferred referencing style.

Pickering, J. E. (2026). Recursive Least Squares for Online Parameter Estimation. ACE-Lab. Teaching Notes: ACE-Lab Learning Guide, Version 1.0. Zenodo. https://doi.org/10.5281/zenodo.23671518